Day 34 Lab — File Integrity Verification with SHA-256

LINUX / KALI OPTION

1. Create a local test file:

   echo "Jayvik Labs Day 34 Integrity Lab" > day34_original.txt

```
┌──(kirthikaa㉿kali)-[~/Desktop]
└─$     echo "Jayvik Labs Day 34 Integrity Lab" > day34_original.txt

┌──(kirthikaa㉿kali)-[~/Desktop]
└─$ cat day34_original.txt
Jayvik Labs Day 34 Integrity Lab
```

2. Calculate SHA-256:

   sha256sum day34_original.txt

```
┌──(kirthikaa㉿kali)-[~/Desktop]
└─$     sha256sum day34_original.txt
bc04e4ce99455d1aa7677be5ce79f07e20037f43d70fa05db77416115fa16a30  day34_original.txt
```

3. Record the result (original hash).

4. Make a small authorized local modification:

   echo "Modified" >> day34_original.txt

```
┌──(kirthikaa㉿kali)-[~/Desktop]
└─$     echo "Modified" >> day34_original.txt
```

5. Calculate again:

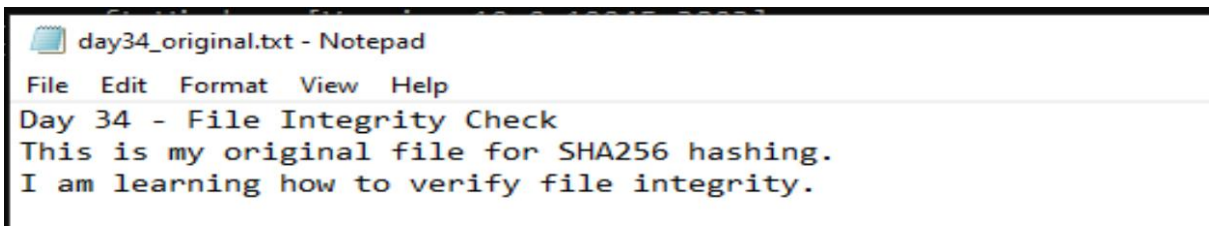
   sha256sum day34_original.txt

```
┌──(kirthikaa㉿kali)-[~/Desktop]
└─$ sha256sum day34_original.txt
46dbe19421721a17042be15f53b4fb7157b17ebad9435dc687a9375e996e84f3  day34_original.txt
```

6. Compare Hash Before vs Hash After — they should be different: YES, both the hashes are different.

WINDOWS OPTION

1. Create your own local text file day34_original.txt.



2. Run: certutil -hashfile day34_original.txt SHA256

3. Record the original hash.

```
C:\Users\Kirthikaa\Desktop>certutil -hashfile day34_original.txt SHA256
SHA256 hash of day34_original.txt:
a5eba6eab4662efacc6bdac7e72c158d00ef6a2e44becf8adf5208798dcbbcbc
CertUtil: -hashfile command completed successfully.
```

4. Modify the text file.

5. Run the command again.

```
C:\Users\Kirthikaa\Desktop>certutil -hashfile day34_original.txt SHA256
SHA256 hash of day34_original.txt:
2f224908ae44cb35b75d718d2b21702e7ed04602ddd641dafba8266cec165fac
CertUtil: -hashfile command completed successfully.
```

6. Compare the hashes: Yes, they both are different.

LAB TASKS [Task 1 to Task 5 completed]

Task 6: Explain why the hashes changed.

- A hash is generated based on the contents of a file. When the text file is modified, its contents change, resulting in a different SHA256 hash.

Task 7: Explain how a software publisher could use hashes for download verification.

- The software publisher generates a SHA256 hash of the original software file.
- The publisher shares the hash on its official website alongside the download.
- Users calculate the hash of their downloaded file and compare it with the published hash. If both match, the file is likely unchanged.

Task 8: Explain why the expected hash must itself come from a trusted source.

- Attackers could replace both the downloaded file and its published hash with malicious versions.
- If the expected hash comes from an untrusted source, users cannot reliably confirm the file's integrity. Users should obtain hashes from trusted official websites or digitally signed sources to reduce the risk of tampering.